

読む技術・書く技術・伝える技術

15年続けて分かった持続可能なオープンソース開発

azu (@azu_re)

YAPC::Fukuoka 2025

自己紹介

azu

- GitHub: @azu
- 2011年から14年間オープンソース活動を継続
- 3つのプロジェクトを運営中



今日話したいこと

技術を使って、持続可能性を実現する

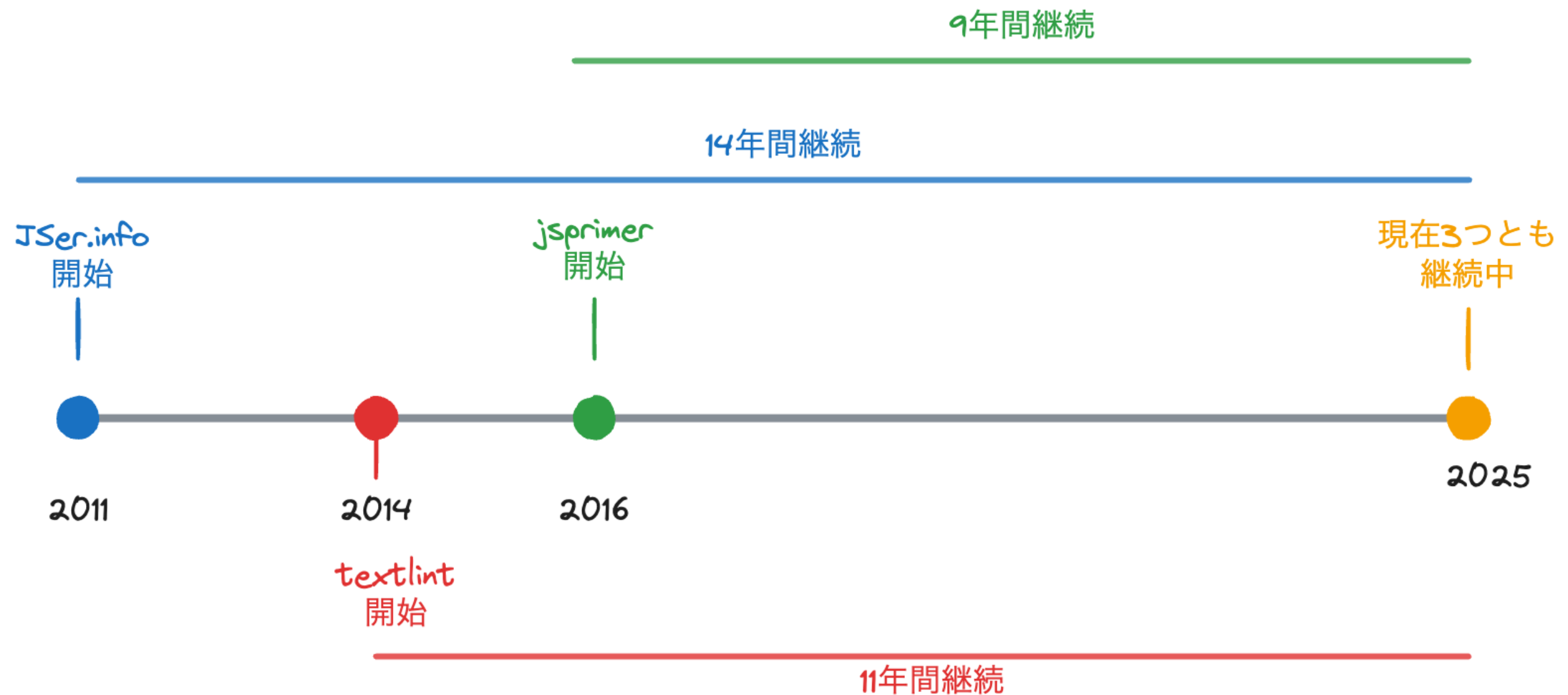
14年間JSer.infoを続け

11年間textlintを続け

9年間jsprimerを続けてきた

その方法をお話しします

3つのプロジェクトのタイムライン



それぞれのテーマ

- **JSer.info:** 読む技術（情報収集と発信）
- **textlint:** 書く技術（品質の自動化）
- **jsprimer:** 伝える技術（協働と設計）

問い: なぜ15年も続けられたのか？

これらのプロジェクトの特徴

一つのアウットプットでは目的が達成できない

- JSer.info: 継続的な情報発信でエコシステムを把握
- textlint: あらゆる言語のlint、ルールとエコシステムの構築
- jsprimer: JavaScriptの進化に追従し続ける

ある程度長くやらないと達成できない目的

長期的な目的を持っている

これらのプロジェクトは
作ったもの（アウトプット）ではなく
実際の影響（アウトカム）を目指している

アウトプット vs アウトカム

アウトカムが評価されるまでには時間がかかる

アウトプット

作ったもの

(記事、ツール、書籍)

アウトカム

実際の影響・成果

(信頼、エコシステム、教育)

この抽象的な区別がOSSで特に
重要になる理由を次で具体化
します。

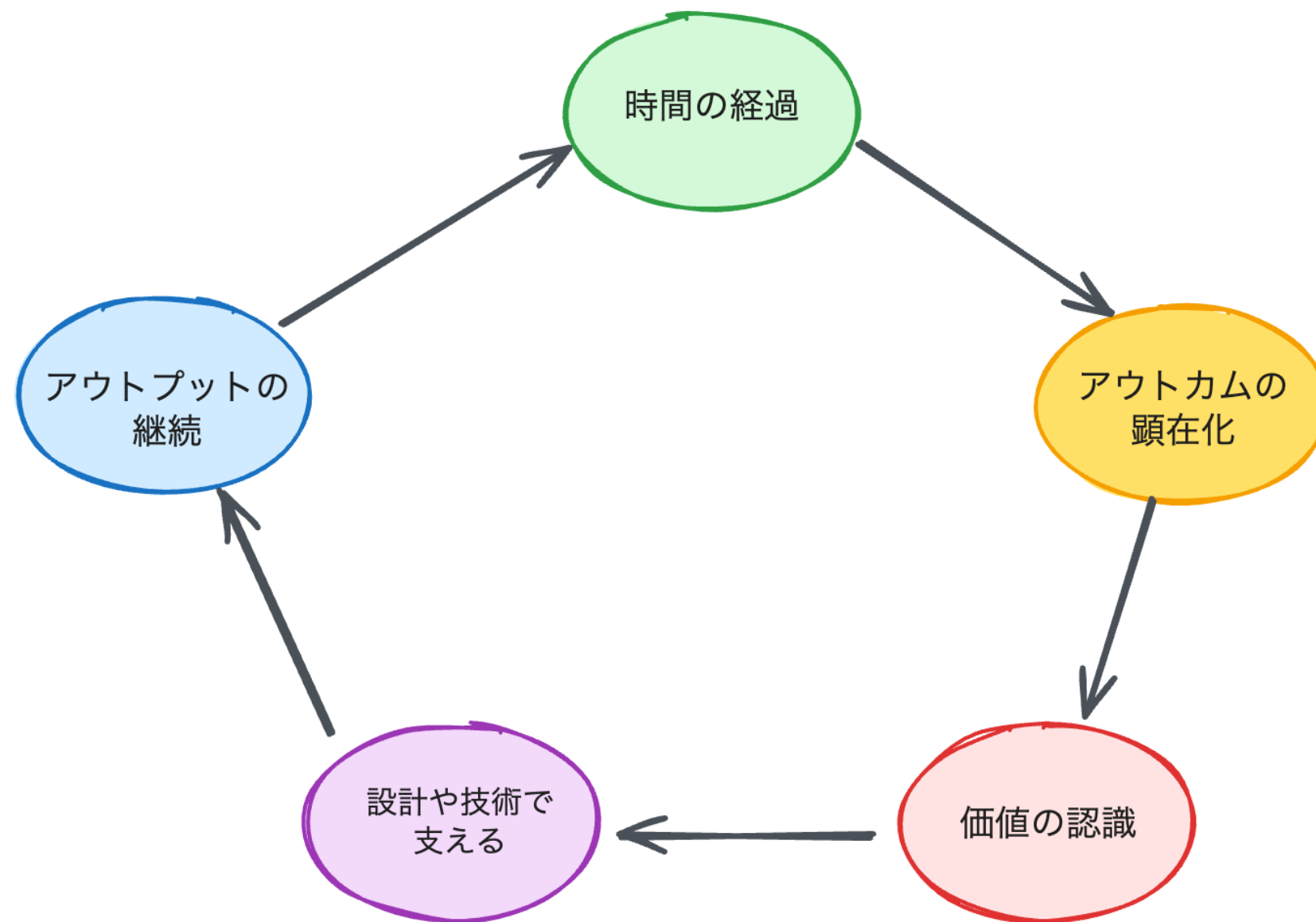
オープンソースにおけるアウトカム

時間をかけて現れるもの

- ツールの普及と影響
- コミュニティの形成
- 知識の蓄積と共有
- エコシステムの成長

これらはすべて、時間をかけないと現れない

持続可能性が必要な理由



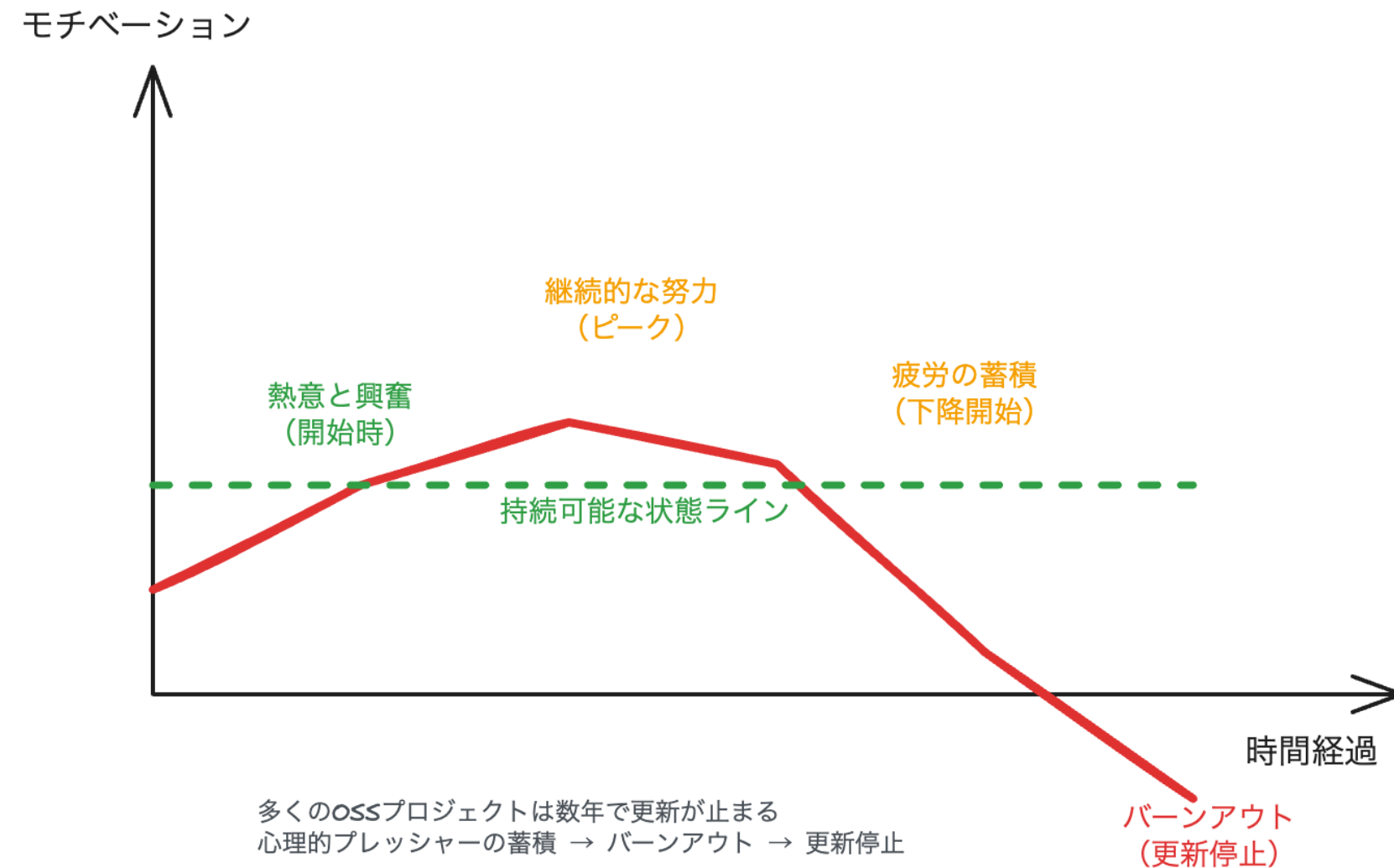
多くのオープンソースプロジェクトの現実

この裏では、同じ期間を走り切れない多数のプロジェクトがあります。

- 数年で更新が止まる
- メンテナーの燃え尽き
- 心理的負担の増大

なぜ燃え尽きるのか？

→ 期待と現実のギャップがある



Burnout

Burnout is the experience of being pulled between expectation and reality at work.

— *Jonathan Malesic, "The End of Burnout"*

仕事における期待と現実の間で引き裂かれる経験

期待と現実のギャップ

期待

- 「自分がやらなければ」
- 「全てに対応すべき」
- 「常に最新に保つべき」

現実

- 時間がない
- 対応できない
- 追いつけない

**→ このギャップがBurnoutを
引き起こす**

問い： どうやって15年間、燃え尽
きずに継続できたのか？

デスマ

技術的依存を増やし

心理的依存を減らす

技術的依存とは

定義：

- 自動化できる部分は自動化する
- ツールや(エコ)システムを育てる
- 再現可能なプロセスを構築する

特徴：

- スケールする
- 疲れにくい
- 他者も利用できる

心理的依存とは

定義：

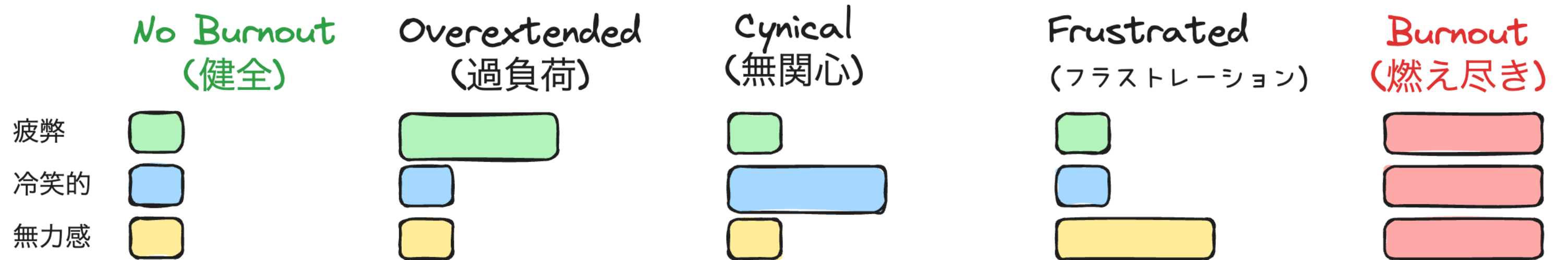
- 「自分がやらなければ」というプレッシャー
- 完璧主義
- 義務感

特徴：

- スケールしない
- 疲れやすい
- 燃え尽きにつながる

Burnoutは真偽値ではない

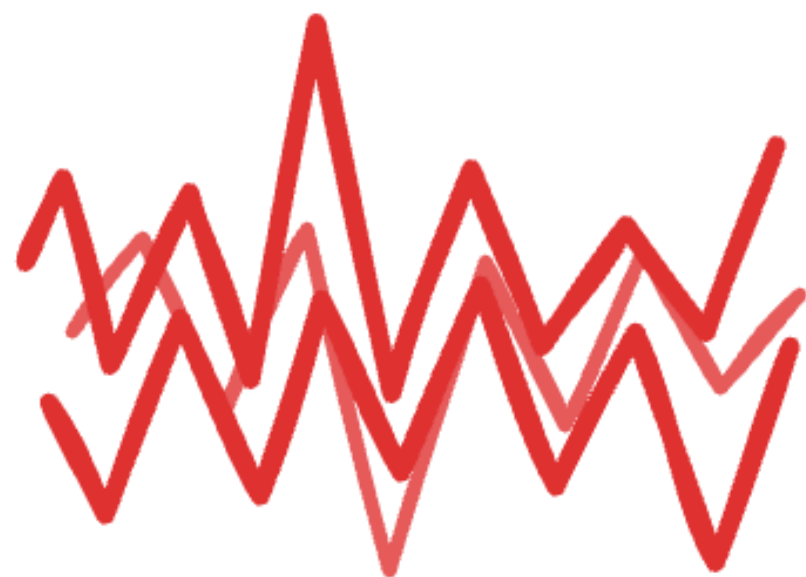
Burnoutは真偽値ではなく、複数の状態の組み合わせで現れる



依存をなくすのは難しい

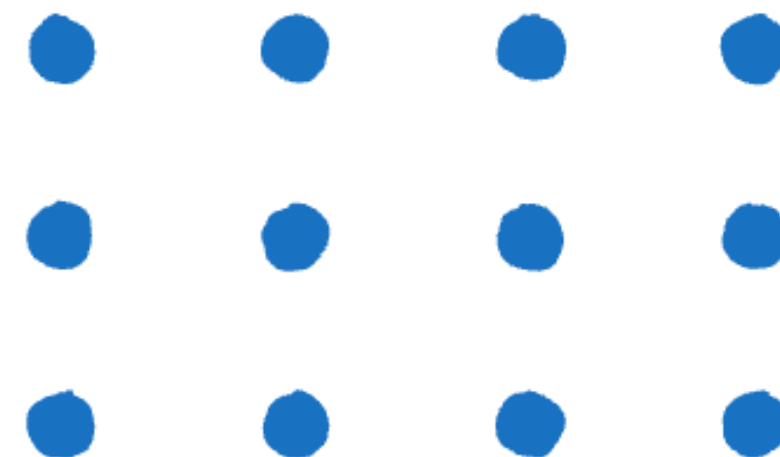
- {技術的, 心理的}依存は「どちらか一方」ではなく、バランスの問題
- 心理的依存を技術的依存に変換できる部分は存在する
-  技術的に解決できない問題は存在する

心理的依存と技術的依存



心理的依存

コントロール困難



技術的依存

コントロール可能

なぜこのバランスが重要か

心理的依存

- コントロールが難しい
- スケールしない
- バーンアウトリスク

技術的依存

- コントロールしやすい
- スケールしやすい
- メンテナンスコストがある

技術的依存を増やし、心理的依存を減らす

- 心理的依存は自覚も難しく、コントロールも難しい
- 技術的依存は設計次第で、何に依存しているのかが明確
- 全てを技術的依存に変えることは不可能だが、
- → 技術的依存を増やし、心理的依存を減らす設計が重要

今日の構成

1. **JSer.info** - 読む技術

→ データドリブン、退屈な部分の自動化

2. **textlint** - 書く技術

→ No core rules、エコシステム

3. **jsprimer** - 伝える技術

→ Living Standard、既知→未知

Part 1

読む技術

JSer.info

JSer.info の目的

整理されたデータである「情報」を伝えること

2011年1月16日創刊

- 750記事、14年継続
- 12,429サイト紹介
- 一度も欠かさず週刊配信



JSer.info

JavaScriptの最新情報を紹介する週刊ブログ

日本語 | 한국어

リアルタイム版は[Realtime JSer.info](#)

投稿一覧

[2025-10-27のJS: Next.js 16、RedwoodJS GraphQL → CedarJS、Vitest 4.0](#)

JSer.info #753 - Next.js 16がリリースされました。Next.js 16 | Next.js Node.js 18のサポート終了、AMP、next lint、middleware.tsの削除など破壊的...

2025年10月27日

[2025-10-16のJS: Bun 1.3、Next.js 16\(beta\)、Node.js 25](#)

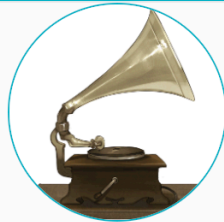
JSer.info #752 - Bun 1.3がリリースされました。Bun 1.3 | Bun Blog ローカルサーバのHot Reloadの改善、HTMLファイルの実行をsingle-file executableへの...

2025年10月16日

[2025-10-10のJS: React 19.2、React Foundation、Birth of Prettier](#)

JSer.info #751 - React 19.2がリリースされました。React 19.2 – React 新しいコンポーネントとして<Activity />コンポーネントが追加され、Hooksとしてuse...

2025年10月10日

 [azu](#)



 [SlackでJSer.infoを購読する](#)

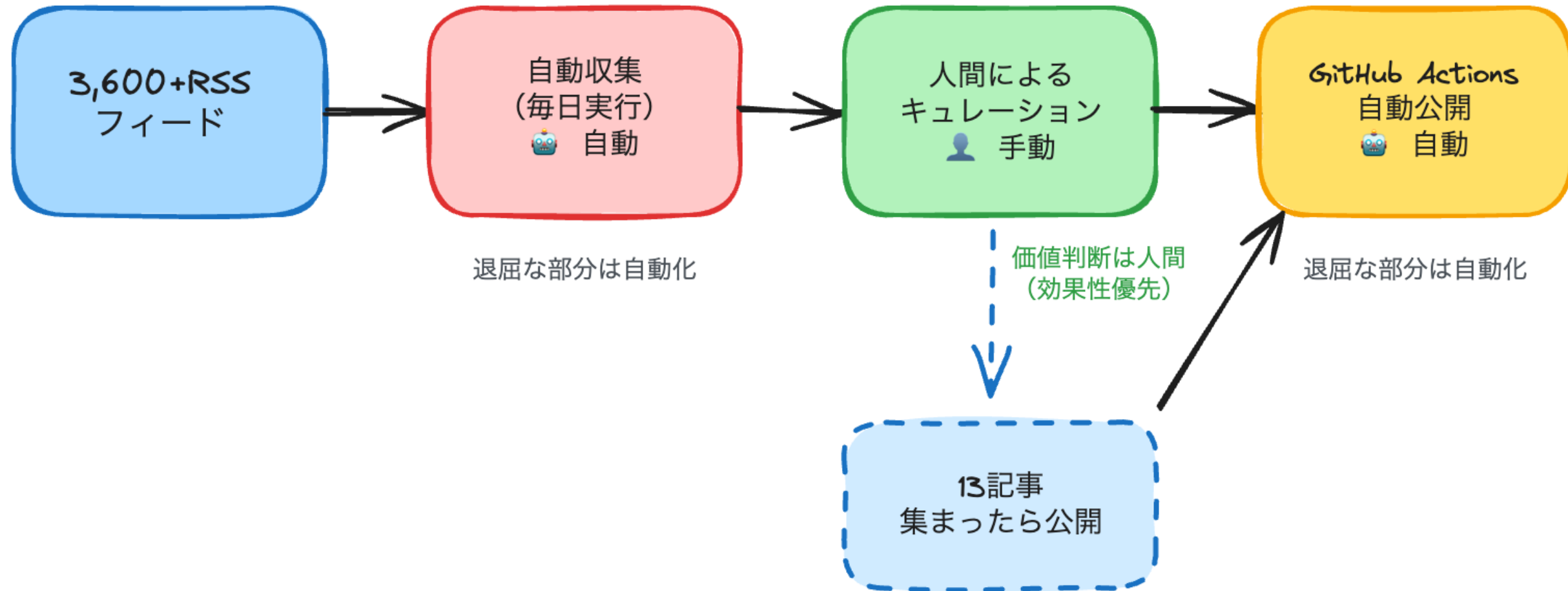
Enter Search

Q

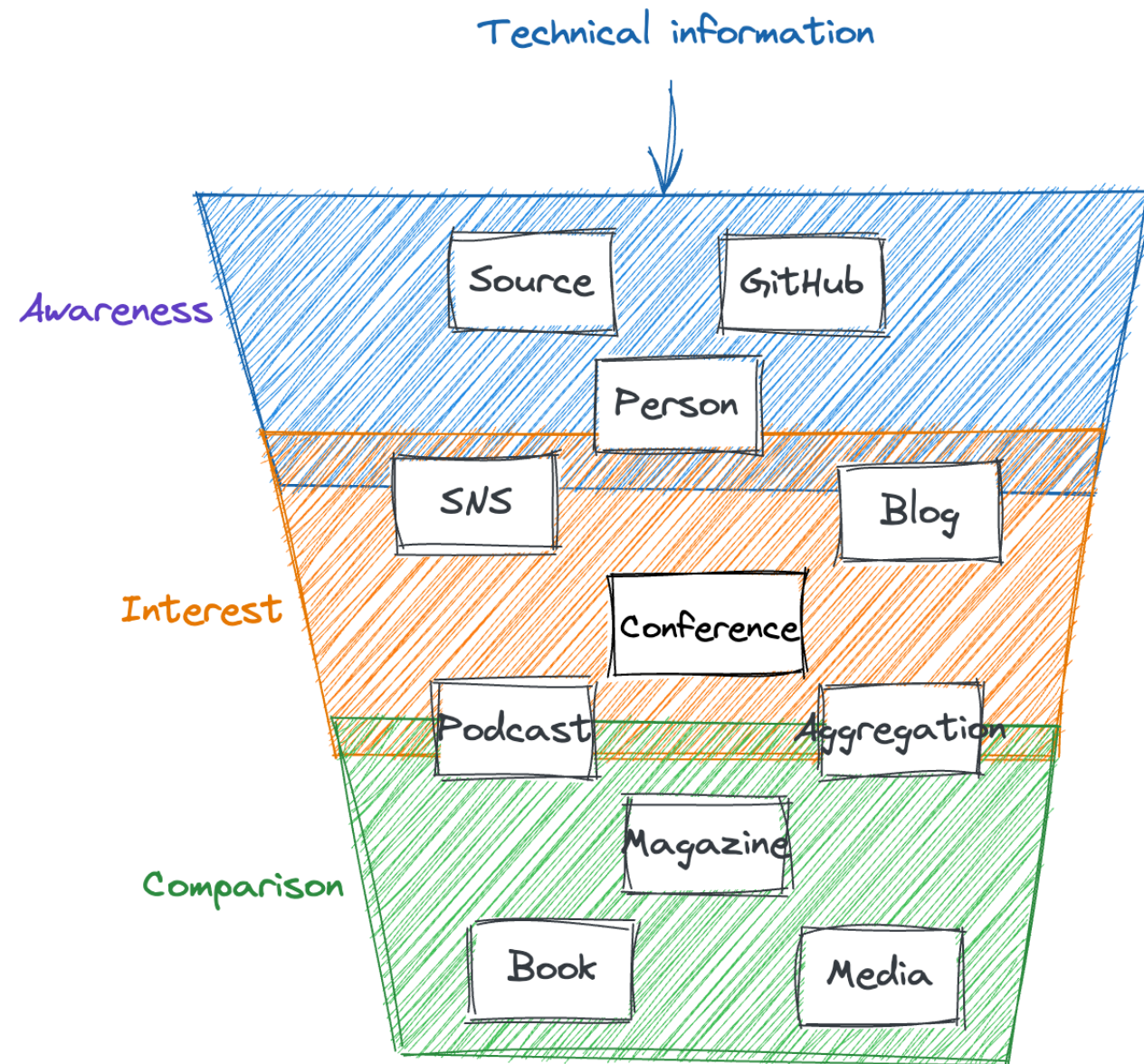
 JSer.info Slackに参加する

情報収集システムの全体像

JSer.info 情報収集システムの全体像



情報のレイヤー分類



3つのレイヤーから情報を収集：

- Awareness層: 新規公開直後の情報（GitHub中心）
- Interest層: トレンドと話題性（SNS、アグリゲーション）
- Comparison層: 比較検討された実践的情報（書籍、メディア）

出典: [JSer.info](https://jsr.info/) 10周年

情報収集の具体的な方法

RSSリーダーを基盤に約3,600フィード：

- GitHub（全体の30-40%）
- Trending、リポジトリ新着、リリース情報
- 日次・週次ニュース
- JavaScript Weekly、Node Weekly
- アグリゲーション
- daily.dev、Echo JS、はてなブックマーク
- 個別ブログ・サイト
- Qiita、Medium、dev.to

受動的収集の利点: フィードは更新されるものに収束

読む上で退屈な部分だけ自動化

自動化する部分

退屈な作業

- RSSベースの自動収集
- 特定の企業やOrganizationを監視
- フィード数は収束する

人間が判断する部分

情報の質

- 曖昧なもの（トレンド）より明確なもの（日付）
- 検索結果より構造化されたソース

JSer.infoは自動化と人間の判断を組み合わせる

データドリブンな公開基準

✖ 従来

「毎週月曜日に公開」

→ 書けない週のプレッシャー

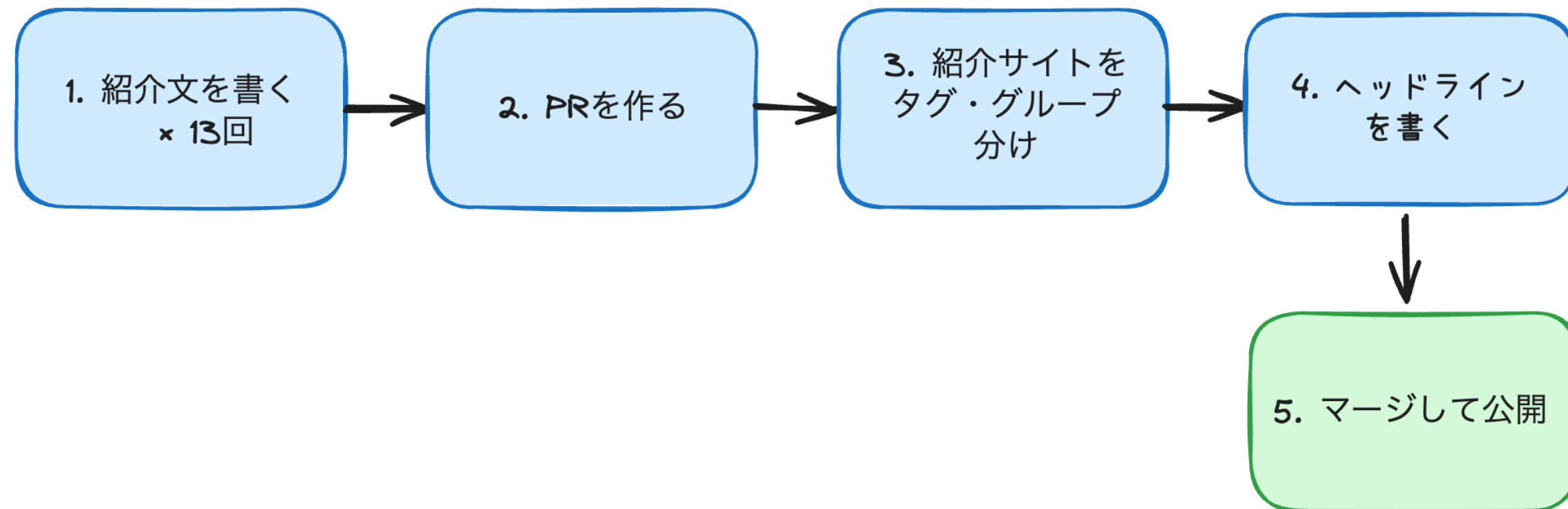
✔ JSer.info

「13記事集まったら公開」

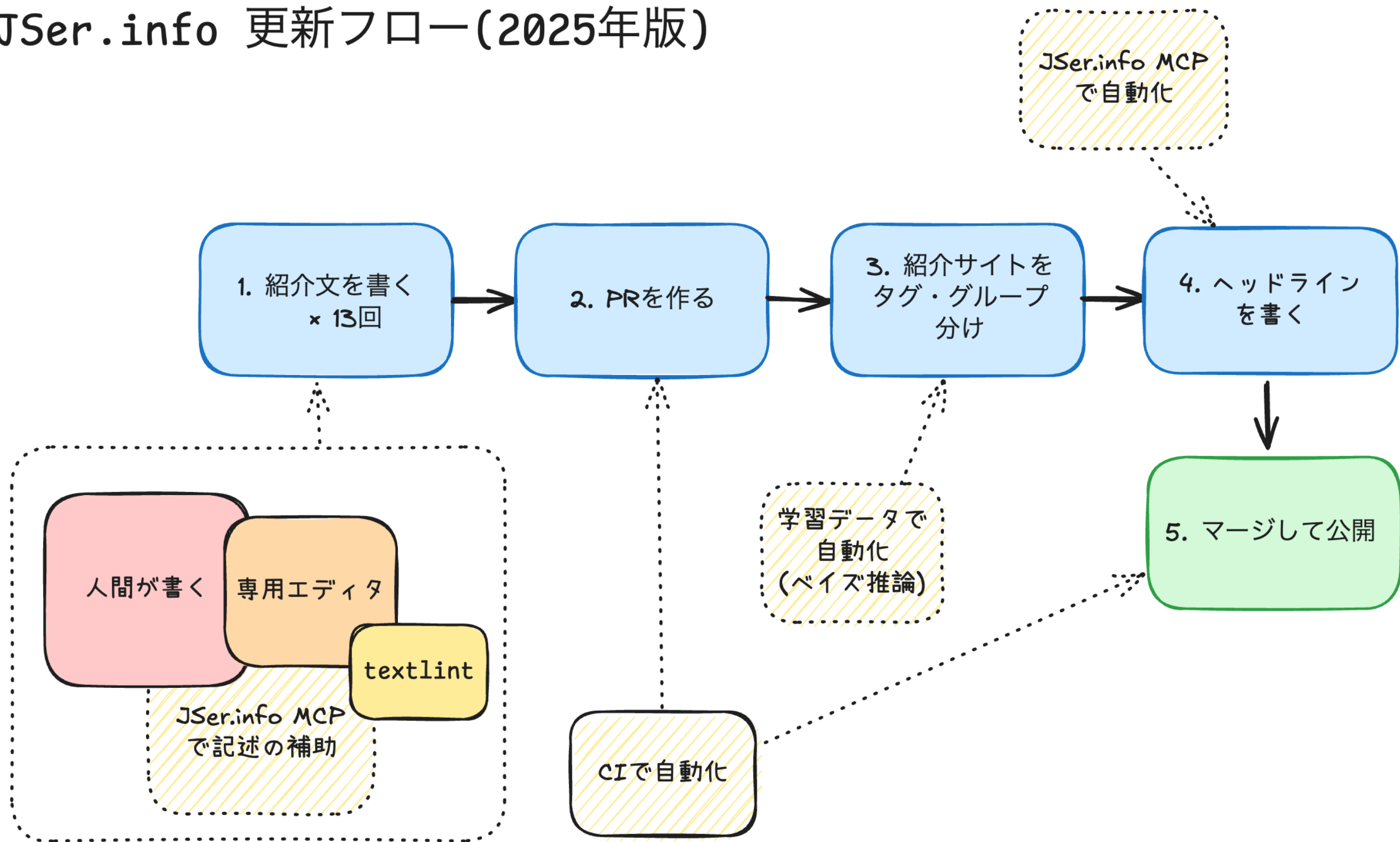
→ 品質でリリース判断

結果: 14年間の継続

JSer.info 更新フロー(2011年)



JSer.info 更新フロー(2025年版)



自動化の詳細

✓ 紹介文を書く

- 専用エディタ(Postem) (2019～)
- MCP執筆補助 (2025～)

✓ PRを作る

- GitHub Actions (2019～)

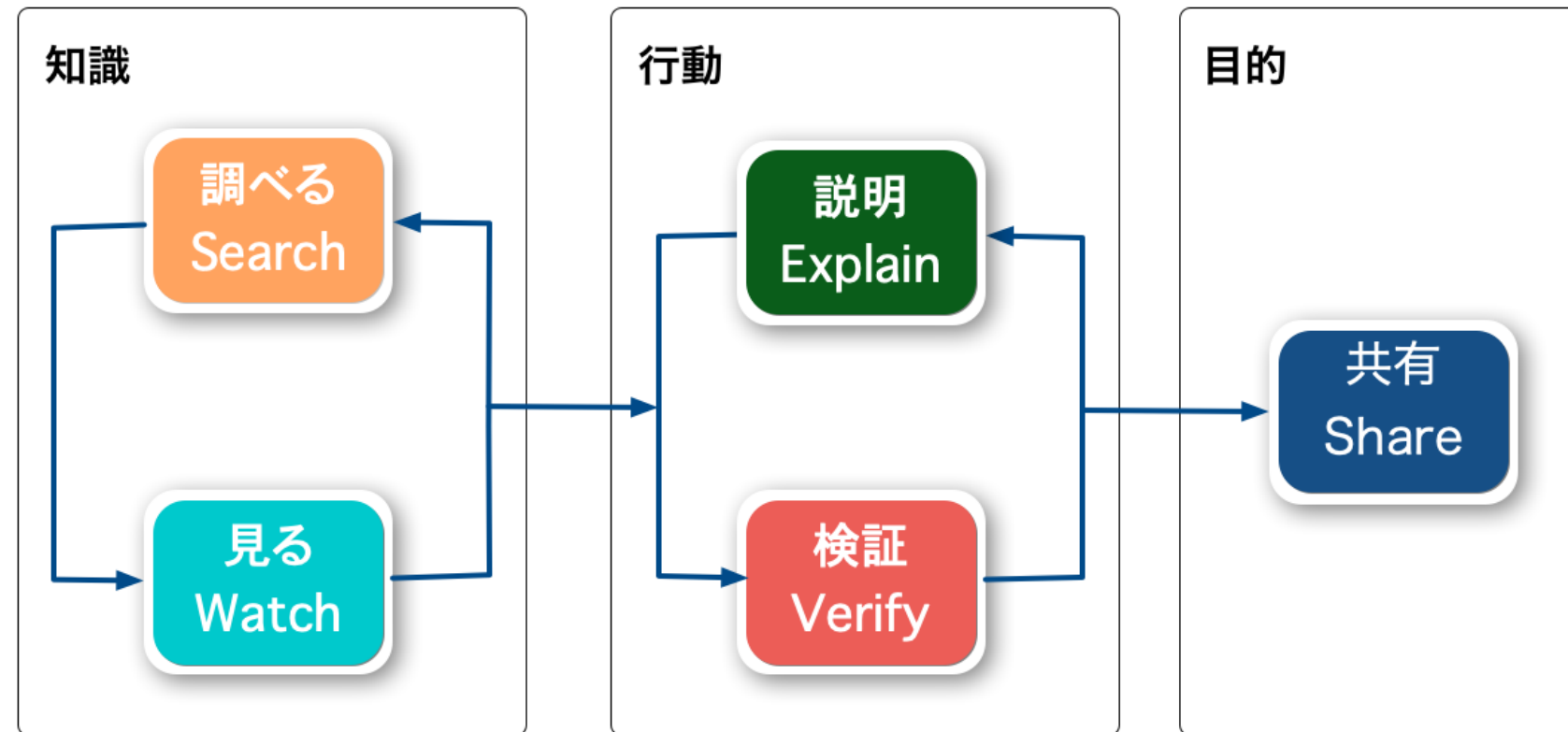
✓ タグ・グループ分け

- タグ推論 (2023～)
- グループ学習 (2020～)
- ヘッドラインをAIで書く (2025～)

段階的に自動化を進めて継続性を確保

JSer.infoでは人は情報の 整理に集中する

1. 見る
2. 調べる
3. 検証する
4. 説明文を書く



JSer.info Policy

「技術的な嘘はつかない」

使わない言葉：

"is Dead"、最強、熱い

慎重に扱う情報：

ベンチマーク数値（マイクロベンチマークは文脈次第で誤解を招く）

使う言葉：

代替方法、特徴、比較

目的は「紹介」ではなく「知ってもらおう」

紹介

情報を提示して終わり

知ってもらおう

読者が元サイトにアクセス

JavaScriptエコシステム全体を
理解

**JSer.infoは目的地ではなく、
導線**

続けることで楽になる循環

継続する



データが蓄積



自動化の材料が増える



自動化が進む



作業が楽になる



続けやすくなる



継続する（最初に戻る）

技術的依存は、継続によって育つ

エポックメイキング

- 振り返って一番大きかったのは、日付ベースではなく記事数ベースの公開基準にしたこと
- この13記事という基準は初めてから4-5年ぐらい経って、中央値を見るとここに安定していたことに気づいた
- この発見で心理的なプレッシャーが大きく軽減できて続いたと感じている
- <https://github.com/jser/status-of-post>
- <https://jser.info/status-of-post/>

JSer.infoまとめ

技術的依存を増やした：

- データ基準（13記事で公開判断）
- 退屈な部分の自動化（収集、PR作成、分類）
- 継続による技術的依存の改善（蓄積 → 自動化 → 継続）

心理的依存を減らした：

- 週次固定締切（毎週月曜のプレッシャー）を廃止
- JSer.info Policyを作り、基準を明確化

Part 2

書く技術

textlint

読むから書くへの移行

読む技術の洞察が書く技術の改善をどう促したかを次で示します。

きっかけ：

- JSer.infoで情報を読み続けていた
- 「読むだけでは物足りない」
- 「自分でも書いてみよう」

2014年3月： [JavaScript Promise](#)本の執筆開始

書くことで気づいたこと

書くと、読む量が増える

なぜ？

- 調べる（情報を読む）
- 検証する（ドキュメントを読む）
- 自分が書いたものを読み返す

書くことは、読むことの延長

書く中で見つけた課題

JavaScript Promise本執筆時の問題：

- 表記揺れが気になる
- 「JavaScript」 vs 「Javascript」
- 「コールバック」 vs 「callback」

解決策： ツールを作ろう

textlint

- 自然言語(日本語や英語など)に対するLinter
- MarkdownやHTMLなどのマークアップ言語に対応している
- ビルトインのルールは0
- 利用できるルールは100以上ある
- CI/CDに組み込める自然言語のチェッカー(表記揺れ、スペルチェック、誤用、読みやすさのチェックなど)

textlintの考え方

「自然言語には正解がないため、
拡張の柔軟性を持つこと」

コア原則：

- ルールをコアで持たない
- プラグインで拡張できるようにする

No core rules戦略

「楽に使える」より「適切に使える」

従来のLinter

インストール

→ デフォルトルール

→ すぐ使える

textlint

インストール

→ 何も起きない

→ ルール選択

コアとルールの分離

- ESLint/RuboCop/PyLintなどはコアにルールを持っている
- コアにルールを持つとルールのIssueもコアに集まる = 心理的負担が集中する
- コアでやらないことではっきりさせることで、責任を分離できる
- そもそもあらゆる自然言語に対応したルールをコアに持つことは不可能
- textlint - Linterの作り方

コアとルールの分離の結果

- ルールは300以上
- エディタの拡張や言語サポートもコアから分離
- 責務が分散するとメンテナンスも分散される
- コアから切り離せると、ルールの追加とかはやりやすくなる
- <-> 一方で、ルールが個人に紐づくメンテナンスが止まる確率は高くなる
- → ルールをOrganizationに集めるなど、緩いまとまりを持ったコミュニティを作ることによってバランスをとっている

11年続けたらAIがユーザーになった

- プラガブルの弱点は、ユーザーがプラグインを選択しないといけない点
- 人間が選択するのは大変
 - ルールによって求める文章の質が異なるので、目的に合ったルールを選ぶのが難しい
 - ちょうど良い使い勝手を実現するのが難しい
- → 2023年以降、AIがユーザーになっていくことでこの難しさが変化

textlintのMCP統合

2025年1月：textlint v14.8.0でMCPサーバー対応

```
npx textlint --mcp
```

- AIアシスタントから直接呼び出し可能
- VS Code、Cursor、Claude Code統合

AIベースライン時代の品質管理

従来（～2022年）

- テクニカルライティングのルールなどは文章を書くのに慣れていないユーザーにとっては難しい
- → 意識して書かないとエラーを避けるのは難しい

AI時代（2023年～）

- AIはルールのエラーメッセージを理解して瞬時に修正できる
→ 人間は書くことに集中できる
- 一方で、新しいAI固有のエラーが増える
→ [textlint-rule-preset-ai-writing](#)を作成

@textlint-ja/textlint-rule-preset-ai-writing

AI特有の文章構造を検出するtextlintルールセット

- 絵文字の多用 -     
- 過剰表現、強調の多用 - 革命的、民主化、大幅な改善
- コロンの直後にブロック要素が続く英語的なパターン
- 同意過剰 - あなたは正しい、その通りです

AIも人間もエラーメッセージを読めるように

- Linterはエラーが発生した時にエラーメッセージをドキュメントとして提供する
- 人間もAIもエラーメッセージを読んで理解し、修正する
- エラーメッセージの質が重要になる

エラーメッセージの改善

- 自然言語は不定な部分が多い。実装に落とすのも工夫が必要
- それ以上に、エラーとなった場合のメッセージが難しい
- 例えば「xxxは冗長な表現です」というルールがあった場合に、なぜ冗長なのか？どう直せば良いのか？を説明するのが難しい
- すなわち、実装者にとっても心理的負担が大きい部分がある

人間は柔軟すぎて、文章の質を評価しづらい

- 人間は自然言語をパターンで認識するので数値化が難しい

こんにちは みさなん おんげき ですか？ わしたは げんき です

- そのため、人間がエラーメッセージの質を上げるには量が必要
- 量が必要だと、過剰な負担になる

エラーメッセージの LLM as Judge

- 技術的に改善するにはエラーメッセージの質を数値で評価したい
- LLM as Judgeでエラーメッセージの質を評価する仕組みを作成中
promptfoo
- エラーメッセージの質をLLMで評価し、スコアを元に改善を繰り返す
- 技術的改善を回しやすくする

promptfoo <https://www.promptfoo.dev/>で実装している

promptfoo

Create

▼

Results

▼

Prompts

Datasets

History

Model Audit

🔍 評価（小規模テスト）

▼

🔗 ID: eval-B9I-2025-10-13T12:37:33

📄

✉ Author: Unknown

✎

Zoom

150%

▼

Display

Show all results

▼

Search

🔍 Text or regex

▼

5 results

Columns (3)

Table Settings

Eval actions ▼

	Variables		Outputs
Description	text	errorMessage	
Row #1	私は彼は好きだ	一文に二回以上利用されている助詞 "は" がみつかりました。 次の助詞が連続しているため、文を読みにくくしています。 - 私"は" - 彼"は" 同じ助詞を連続して利用しない、文の中で順番を入れ替える、文を分割するなどを検討してください。	textlint-no-doubled-joshi 100.00% passing (5/5 cases) //{{text}} 🔍 🔗 5 PASS accuracy 1.00 clarity 1.00 contextual_fit 1.00 fix_quality 1.00 fixability 1.00 一文に二回以上利用されている助詞 "は" がみつかりました。 次の助詞が連続しているため、文を読みにくくしています。 - 私"は" - 彼"は" 同じ助詞を連続して利用しない、文の中で順番を入れ替える、文を分割するなどを検討してください。 --- 【修正案】 私は彼が好きです Latency: 5,802 ms
Row #2	あなたは「私は彼は好きだ」と言った。	一文に二回以上利用されている助詞 "は" がみつかりました。 次の助詞が連続しているため、文を読みにくくしています。 - 私"は" - 彼"は" 同じ助詞を連続して利用しない、文の中で順番を入れ替える、文を分割するなどを検討してください。	5 PASS accuracy 1.00 clarity 1.00 contextual_fit 1.00 fix_quality 1.00 fixability 1.00 一文に二回以上利用されている助詞 "は" がみつかりました。 次の助詞が連続しているため、文を読みにくくしています。 - 私"は" - 彼"は" 同じ助詞を連続して利用しない、文の中で順番を入れ替える、文を分割するなどを検討してください。 ---

YAPC::Fukuoka 2025 - azu

57

Showing 1 to 5 of 5 results

Page 1 of 1

Results per page: 10 ▼ Go to: 1

◀ ▶

textlintまとめ

技術的依存を増やした：

- プラガブルアーキテクチャ
- MCP適応/AIの活用

心理的依存を減らした：

- コアとルールの分離 = 心理的負担の分散
- LLM as Judgeによる実装時の負荷軽減(将来)

Part 3

伝える技術

JavaScript Primer

書くから伝えるへの移行

書く過程で浮かび上がった『読者視点』が、伝える技術の枠組みを生み出した。

転換点：

- 個人で書く → 相手に伝える
- 一人で書く → 共著で書く
- **「読者」を強く意識する瞬間**

2016年：JavaScript Primer開発開始

JavaScript Primer (jsprimer) とは

- 2016年から執筆、6つのメジャーバージョンを経て更新され続けている JavaScript 入門書
- 書籍版 (amazon.co.jp/dp/4048931105) も販売中
- ウェブ版 (jsprimer.net) は無料で公開
- GitHub: [asciidwango/js-primer](https://github.com/asciidwango/js-primer)
- 書籍版とウェブ版の内容に違いはなく、オープンソースで開発中

JavaScript Primer 改訂2版

迷わないための入門書

azu, Suguru Inatomi 著

変化に
対応できる
基礎を
身につけよう！



jsprimerの目的

3つを継続的に提供：

1. **書き方**：基本文法などのJavaScriptの書き方
2. **作り方**：アプリケーションの作り方
3. **学び方**：JavaScriptの進化を自分で知る学び方

新しくJavaScriptを学ぶ人が迷わないようにするための入門書

目的を達成するためにとって戦略

- Living Standard戦略
- Design Doc（設計文書）
- 既知→未知の原則
- 自動テストによる品質保証

Living Standard戦略

- ECMAScript ではアクティブな状態と静的な状態を使い分けている
- アクティブな仕様 = Living Standard
 - <https://tc39.es/ecma262> 常に最新のものとして公開される
- 静的な仕様 = Snapshot
 - <https://ecma-international.org/technical-committees/tc39/>
 - ECMA のサイトで1年に一度更新される

アクティブなウェブ版と静的な書籍版

- ECMAScript 仕様のモデルに真似る:
- **Living Standard:** ウェブ版 (jsprimer.net)
 - 常に最新版を維持
- **Snapshot:** 書籍版
 - 安定版として時々リリース(読みやすさ重視)

Living Standardを書籍に使う

- 簡単に言えば、完成前から公開していく
- これによって、完成しないと公開できないという心理的負担を減らす

Design Doc

- 書籍のような文章を書くにも設計が必要
- 書籍は章ごとに分かれているので、各章ごとにDesign Docを作成
- これによって、章の方向性を明確化し、書きやすくなる

jsprimerのDesign Doc構成

各章/

- └─ README.md (本文)
- └─ OUTLINE.md (設計文書)

meeting-notes/

- └─ YYYY-MM-DD.md (ミーティング記録)

OUTLINE.mdの役割：

- なぜこの章が必要か
- 何を学ぶか
- どう教えるか

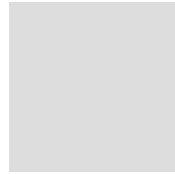
すべての議論を公開：

- Issue/PRでの議論
- Meeting Notesも記録して公開

Design Docの活用

- Design Docがあることで書いてる途中でブレにくい
- 章の目的が明確なので、書き始めるハードルが下がる
- それでも、章の最初の一步は重い
- → そこでAIを活用して最初の一步を軽減

Design Doc → AI生成 → レビュー



Design Doc to AI to Review

- Design Docを元にAIにドラフトを生成させる
- [textlint-rule-preset-ai-writing](#)を使ってAIらしい表現を検出
- 人間がレビューして修正
- 大まかなイメージは作成でき、最初の一步が格段に楽に
- 実際に書いた章: [イテレータとジェネレータ](#)

書籍のレビュー

- 原則: 既知→未知
- textlintで表記揺れや誤用をチェック
- 自動テストでコード例の動作をチェック

既知の言葉で未知を説明する

読者が知っている言葉で説明してから、未知の言葉を導入する

よくない例

「クロージャとは、関数の中に作られた関数が、その外側の関数の変数にアクセスできる仕組みのことです。」

よい例

「関数の中に関数を作ると、その内側の関数から外側の関数の変数にアクセスできます。これをクロージャと呼びます。」

自動テストによる品質保証

8種類のテストを組み合わせで品質を保証

文章チェック

- textlint: 日本語の表記チェック
- ESLint: JavaScriptコードの品質チェック

実行チェック

- Doctest: コード例の出力検証
- Unit Tests: Mochaによるテスト実行
- Executable Code: サンプルコードの実行確認

Doctestによるコード例の検証

Markdown内のコード例を実際に実行して検証

```
// コメントで期待値を記述
```

```
const result = 1 + 1;  
console.log(result); // => 2
```

特徴:

- ドキュメントと実装の乖離を防ぐ
- リファクタリング時の安全性を確保
- 非同期処理にも対応

CI/CDでの自動実行

すべてのテストはPull Requestごとに自動実行

- GitHub ActionsまたはNetlifyでビルド
- textlint、ESLint、Doctestなど全テストを実行
- プレビュー環境で実際のレンダリングを確認

結果: 使っているから続く、続いているから使う

技術から人へ - 持続可能性の課題

技術的依存で解決できたこと：

- 品質保証の自動化（CI/CD）
- Living Standardによる継続更新
- Design Docによる設計の明確化

技術だけでは解決できない

技術的依存で解決できた

- ✓ 品質保証の自動化 (CI/CD)
- ✓ Living Standardによる継続更新
- ✓ Design Docによる設計の明確化
- ✓ Sandpackによる貢献障壁の低下

VS

技術では解決できない

- ✗ 継続的な労力 (年間30日分)
- ✗ コミュニティの維持・活性化
- ✗ モチベーションの維持

だから経済とコミュニティの設計が必要

透明な経済モデル

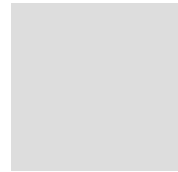
年間コスト試算：

- 約30日分の労力
- 約70万円相当

収入源：

1. 書籍売上
2. Open Collective
3. GitHub Sponsors

Open Collective



Open Collective Dashboard

具体例： <https://opencollective.com/jsprimer>

仕組み：

- プロジェクト単位の資金管理
- 企業向けスポンサー特典
- 透明な収支公開

Fibonacci報酬システム

タスク複雑度：1， 2， 3， 5， 8ポイント

1ポイント = 約\$7

(目安：2ポイント ≒ 1日分の作業)

年間想定：60ポイント分の作業

仕組みの特徴：

- 貢献度の可視化
- 金銭的還元の仕組み化
- コントリビューター参加の促進

100人以上のコントリビューター

どうやって？

- ハードルを下げる
 - OUTLINE.mdで方向性明確化
 - ブラウザで編集可能
 - Contributeも簡単に
- 透明性を保つ
 - すべての議論をGitHub上で
 - Meeting Notesも公開
 - 判断理由の記録

jsprimerまとめ

技術的依存を増やした：

- 継続仕様（Living Standard、ES2025対応）
- 自動品質（CI/CD、3層チェック）
- 設計文書（OUTLINE.md、AI活用）
- 経済透明化（Fibonacci報酬、Open Collective）

心理的依存を減らした：

- 一度で完成要求（継続更新前提）
- 著者を増やす（100人以上のコントリビューター）

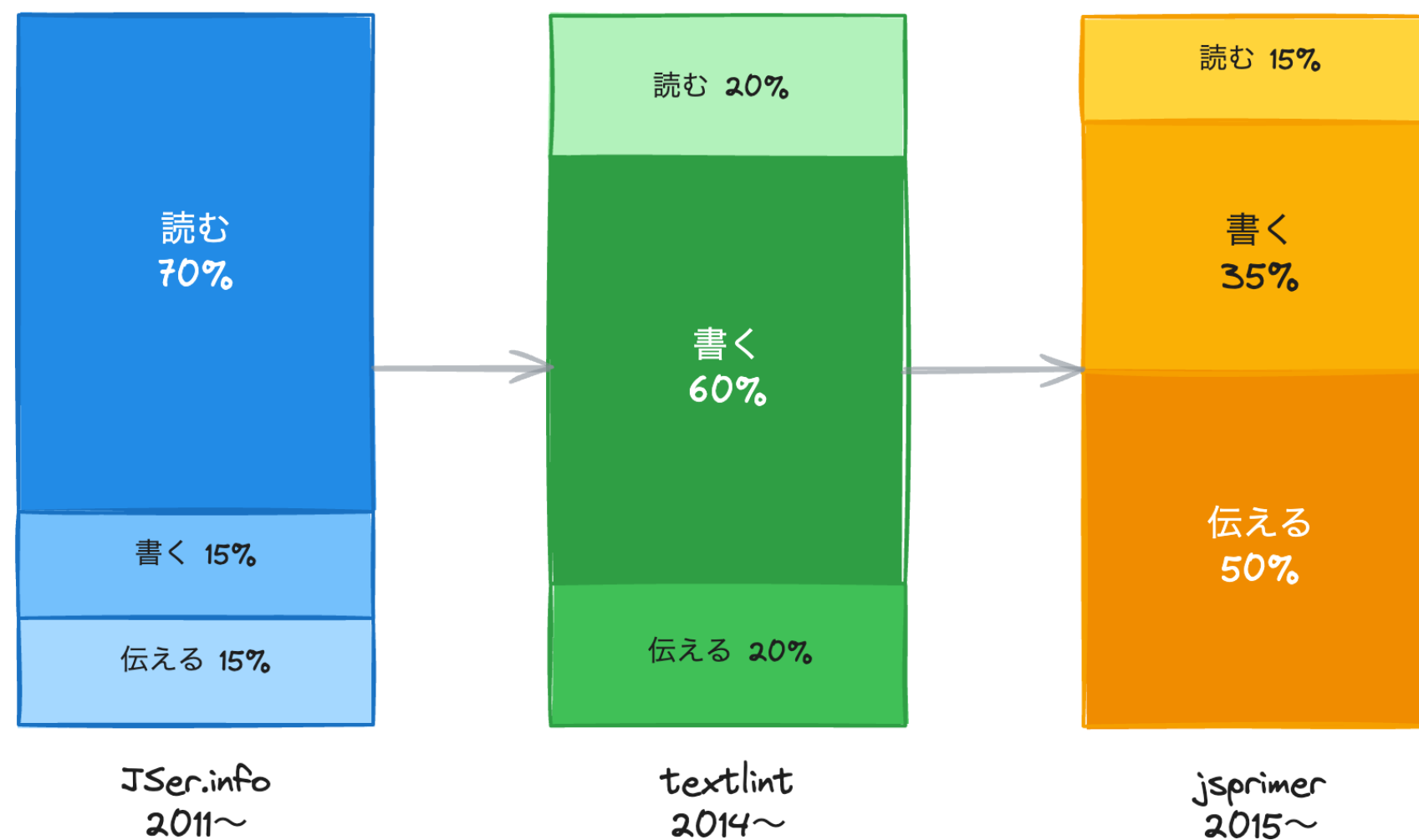
Part 4

循環の技術

複利効果と段階的發展

各プロジェクトの重点配分

読む技術・書く技術・伝える技術



段階的な発展

2011: JSer.info

読むことから始まった



2014: textlint

書くことで読む量が増える体感



2015: JavaScript Primer

伝えることで読む・書くの質が上がる

自然な進化、意図的な設計ではない

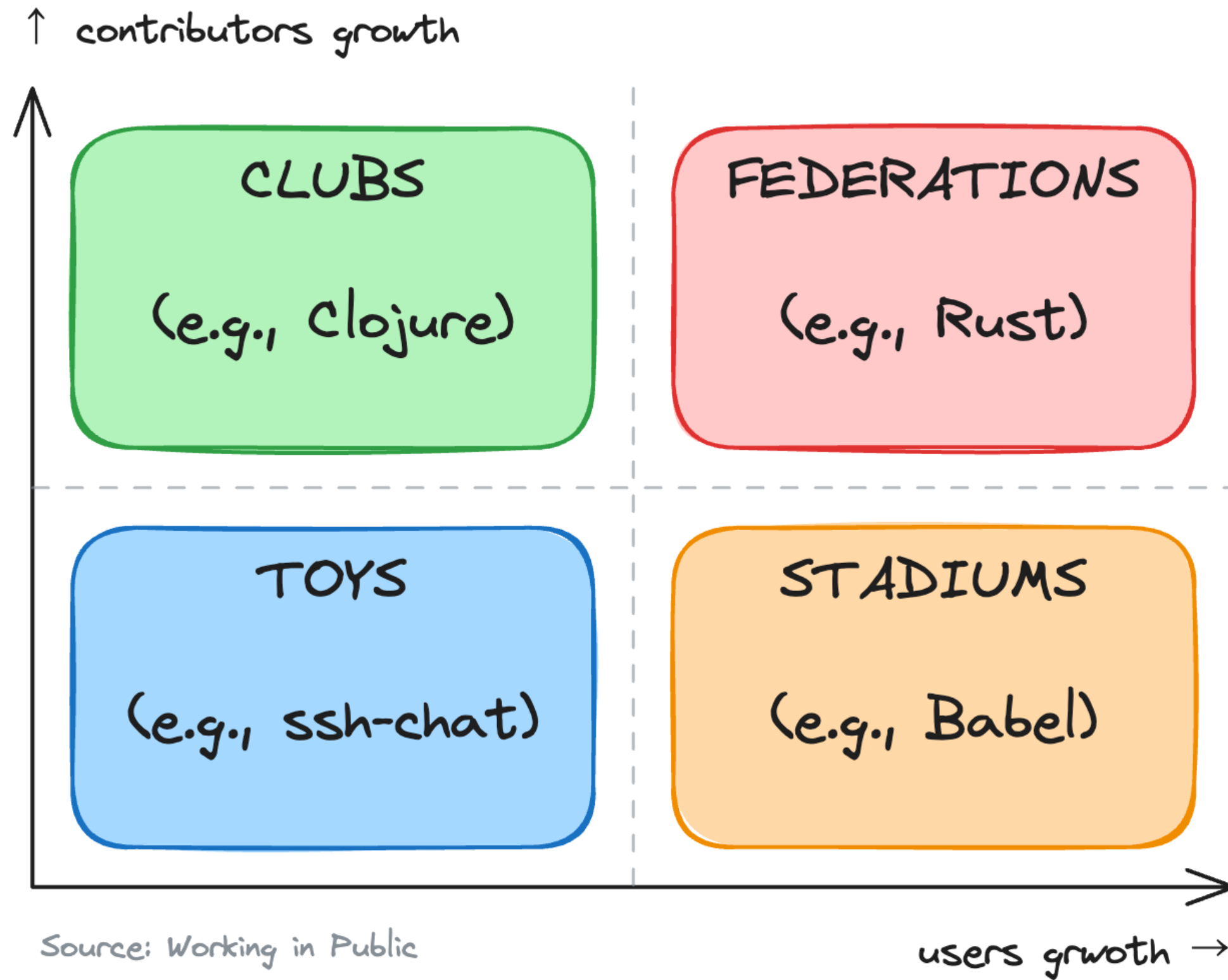
オープンソースではあるがやり方が違う

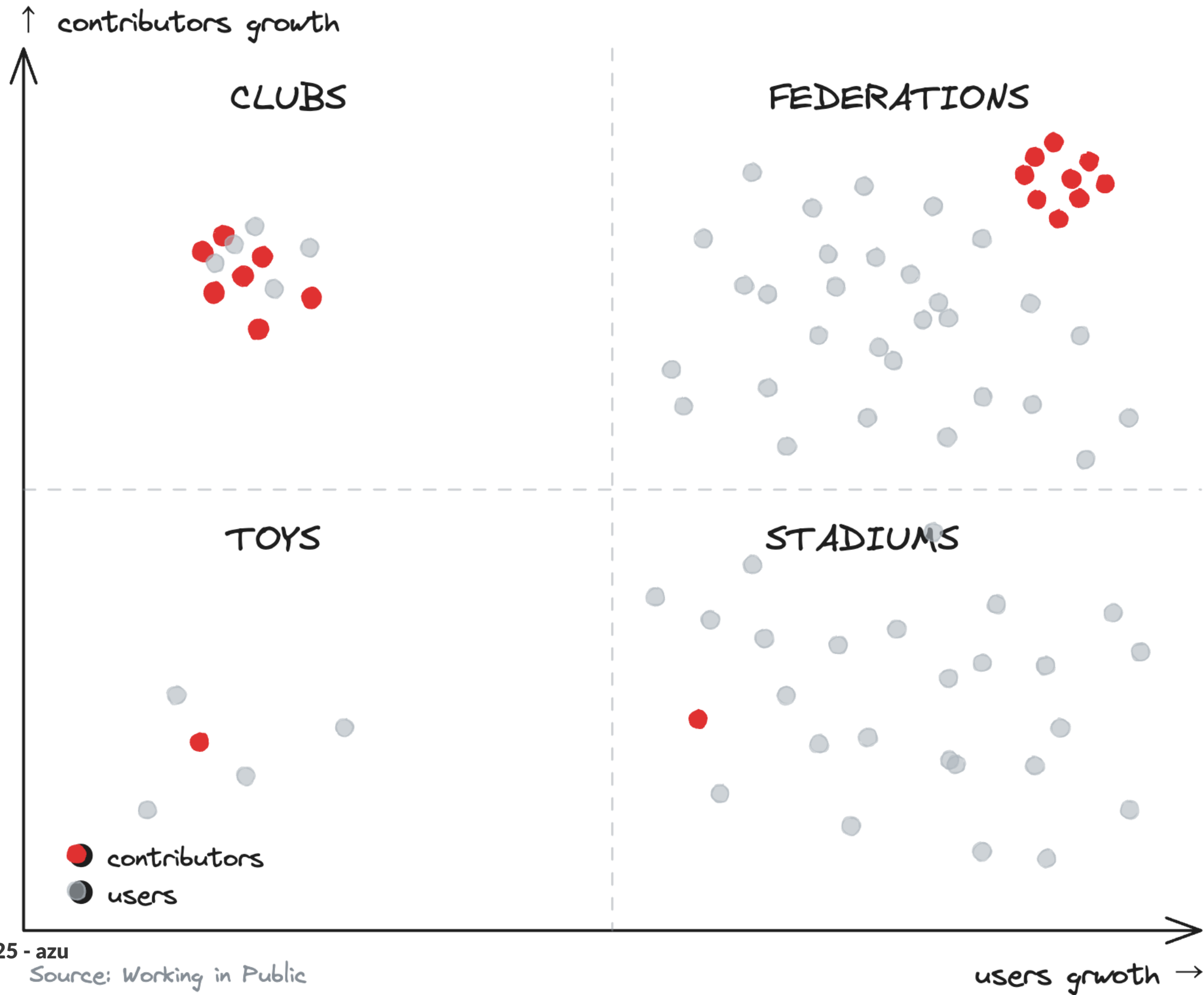
- JSer.info/textlint/JavaScript Primerはどれもオープンソース
- しかし、それぞれのプロジェクトはやり方が異なる

オープンソースの4つのモデル

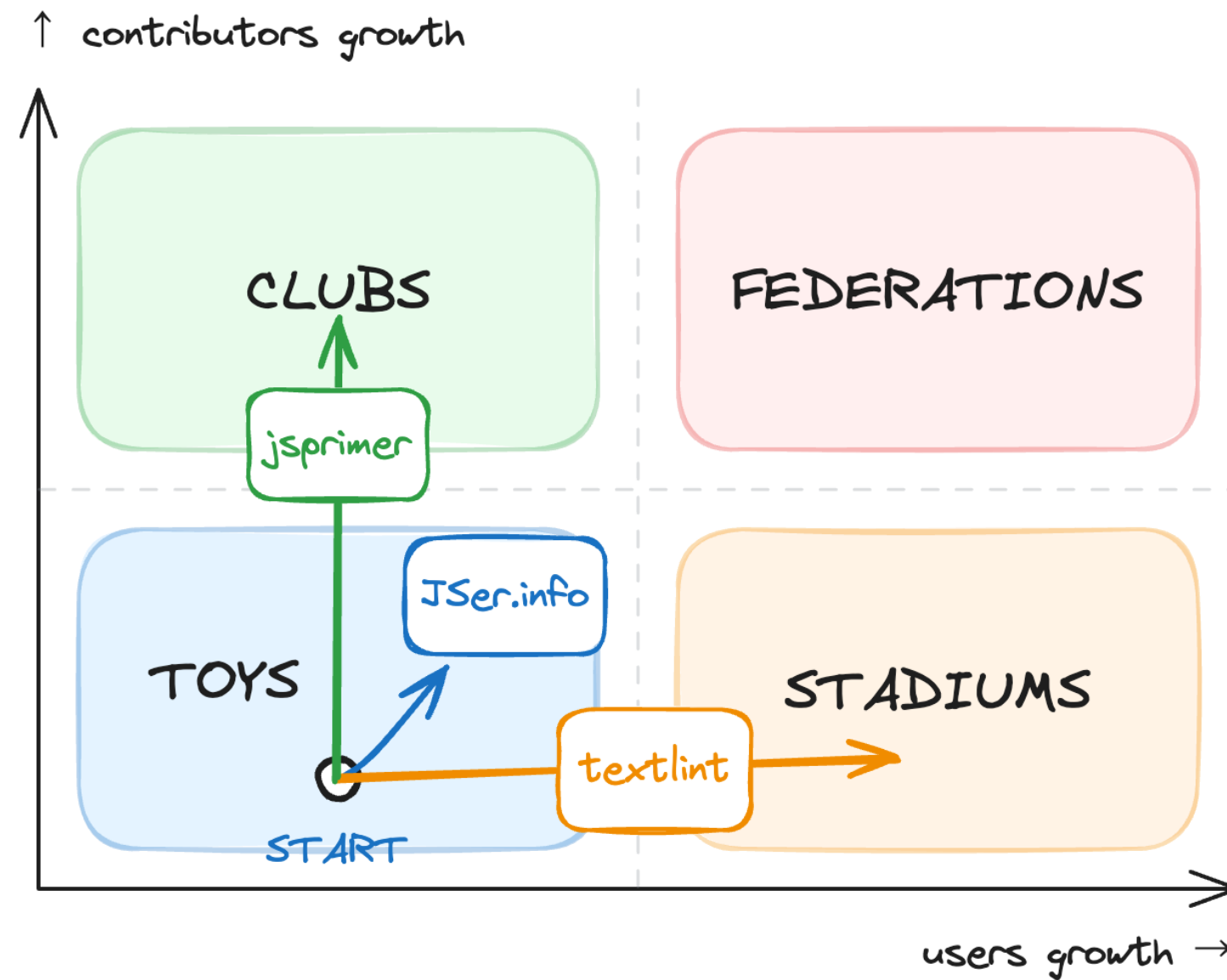
モデル	ユーザー	貢献者	特徴
TOYS	Low	Low	サイドプロジェクト
STADIUMS	High	Low	集中型メンテナンス
CLUBS	Low	High	重複コミュニティ
FEDERATIONS	High	High	複雑なガバナンス

出典: [Working in Public](#)





プロジェクトはToysから始まる



JSer.info: Toysモデルを維持

最適な規模を保つ設計：

- データドリブンな公開（GitHub Actionsによる自動化）
- 趣味であるから続けるという範囲に保つ
- 心理的プレッシャーを発生させない設計

安定した規模を維持することで、心理的負荷を排除

textlint: Stadiumsモデル

週85,000ダウンロード、少数のメンテナー：

- No core rules戦略
- コアを最小化、エコシステムへ委譲
- 100以上のルールをコミュニティが作成

技術的依存は高いが、心理的依存を下げる設計

jsprimer: Clubsモデル

100人以上のコントリビューター：

- Living Standard（常に更新）
- Design Docで貢献のハードルを下げる
- ユーザーと貢献者が重複するコミュニティ

貢献しやすい構造で心理的負荷を分散

各モデルでの心理的負荷への対応

各モデルでの心理的負荷への対応戦略

clubs (左上: コントリビュータ多)

- 貢献のハードルを下げる
- Design Doc/Living Standard
- コミュニティの自律性を促進

Federations (右上: 高成長)

- ワーキンググループによる分散
- RFC/技術評議会による意思決定
- 責任を組織的に分散

Toys (左下: サイドプロジェクト)

- 成長より安定を優先
- データドリブン/自動化で負荷排除
- 心理的依存を発生させない設計

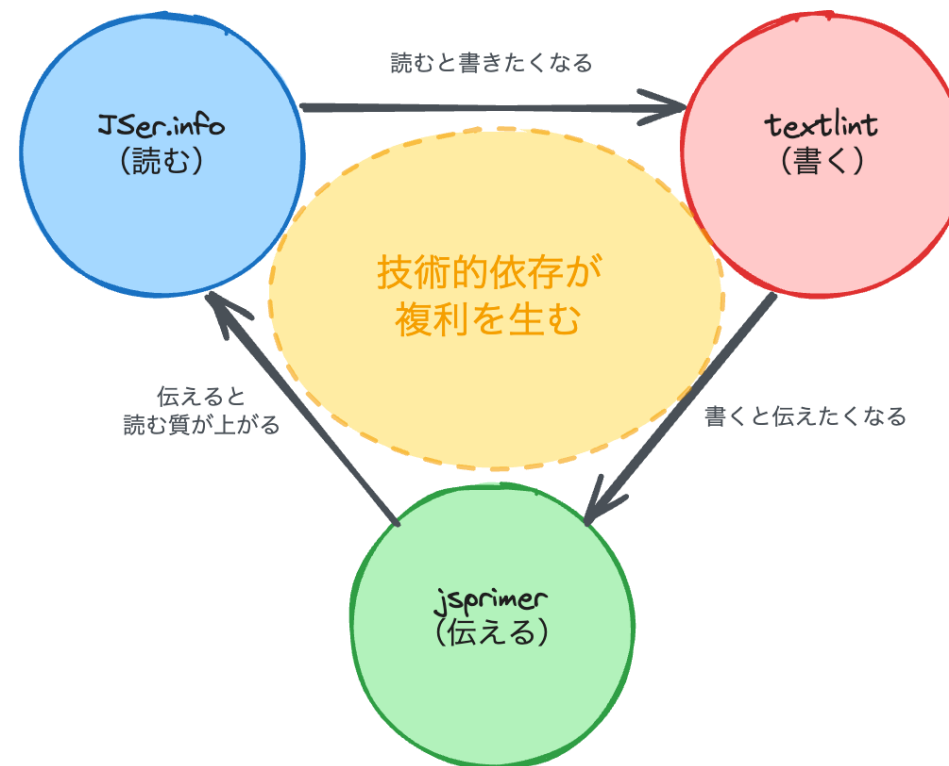
Stadiums (右下: ユーザー多)

- コアを最小化 (例: No core rules)
- エコシステムへの委譲
- メンテナーの負荷を技術的に削減

モデルごとに異なる戦略で持続可能性を確保

相互強化のループ

プロジェクト間の相互強化ループ



プロジェクト間の循環：

読む → 書く → 伝える → 読むの質向上

リスク - 技術的依存の連鎖

どれか一つが止まると全部止まる可能性

これは受け入れているリスク

トレードオフの認識

ただし...

技術的依存は心理的依存と異なり、代替可能性が高い
ツールが止まっても別のツールに置き換えられる

リスクに対する設計上の緩衝

リスクは残るが影響度を小さく保つ設計：

- **再起性:** 復旧手順と自動化により中断から再開しやすい
- **分散:** No core / コントリビューター拡張で責任集中を回避
- **透明性:** 設計文書と経済公開で意思決定の負担を軽減

→ リスクは残るが影響度を小さく保つ設計で継続性を確保

技術的依存と心理的依存

技術的依存 ✓

- スケールする
- 疲れにくい
- 複利効果を生む
- 自動化できる
- **交換可能・差し替え可能**
- **持続がし易い**

心理的依存 ✕

- スケールしない
- 疲れる
- バーンアウトを生む
- 自動化できない
- **コントロールが難しい**
- **持続が難しい**

ま

と

め

12の設計原則

読む技術（JSer.info）：

1. データドリブンな公開基準 [技術]
2. 退屈な部分だけ自動化 [技術]
3. 小さなイテレーション [技術]

書く技術（textlint）：

4. デフォルトルールなし戦略 [心理]
5. 最小コア、豊かなエコシステム [技術]
6. 検証可能なことを自動化 [技術]
7. 最小限の依存関係 [心理]

伝える技術（jsprimer）：

8. Living Standard戦略 [心理]
9. 既知→未知の原則 [心理]
10. 自動テストによる品質保証 [技術]
11. 透明な経済モデル [心理]
12. 使って改善する循環 [技術/心理/循環]

[技術]: 技術的依存を育てる (1,2,3,5,6,10,12)

[心理]: 心理的依存を減らす (4,7,8,9,11,12)

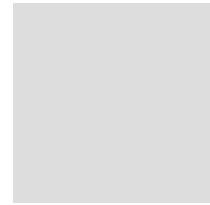
続けることで複利が生まれる

- データセット蓄積
- エコシステム成長
- AI時代への適応力
- プロジェクト間の相互強化

そのための設計原則：

- **技術的依存を積極的に育てる** → スケールする、疲れにくい、複利効果
- **心理的依存を意識的に減らす** → バーンアウト回避、長期継続

Public Writing as Future Asset



15years Public Writing Cycle

- この資料の作り方は、読む書く伝えるの循環をAIを通して行った
- AIがPublic情報を元に読んで、書いて、直し方をつたえて読む
- これができるのってPublicに書いているから

Disclaimer

持続することがなぜ大事?

- この方法がいいというわけではない
- 引き継ぐ難易度が結局高いという問題がまだ未解決
- サプライチェーンの問題もあり、難易度がかえって上がっている

ありがとうございました

- JSer.info: <https://jser.info/>
- textlint: <https://textlint.org/>
- JavaScript Primer: <https://jsprimer.net/>
- GitHub: <https://github.com/azu>

